

Amazon
MS
Google
Facebook
;

Running Median

Getting a stream of integers.
With every new element return the current median.

4	4	→ 4
4 6	4 6	→ 5
4 6 3	3 4 6	→ 4
4 6 3 2	2 3 4 6	→ 3.5
4 6 3 2 9	2 3 4 6 9	→ 4

Approach 1

Maintain data in sorted order
(Insertion Sort)

TC/insertion → $O(N^2)$

total TC → $O(N^3)$

3 7 11 13 14 21 23 27

11 3 13 7 21 23 14 27

$N/2$ smaller
element

↓
max heap

$N/2$ larger
elements

↓
min heap

$N \rightarrow \text{odd}$



median
max heap, top()

$N \rightarrow \text{even}$

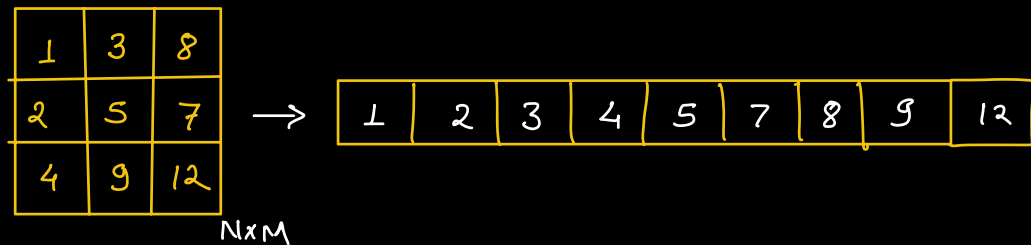


$\frac{\text{max heap, top() + min heap, top()}}{2}$



TC per insertion $\rightarrow O(\log N)$
 Overall TC $\rightarrow O(N \log N)$

Q Given a 2D matrix of size $N \times M$ with sorted rows.
Amazon Merge all the rows into a 1D sorted array.



Approach 1

- Insert all elements in a 1D array. $\rightarrow O(NM)$
- Sort the array. $O(NM \log NM)$

TC: $O(\underbrace{NM}_{\substack{\downarrow \\ \text{No of} \\ \text{elements}}} \log NM)$

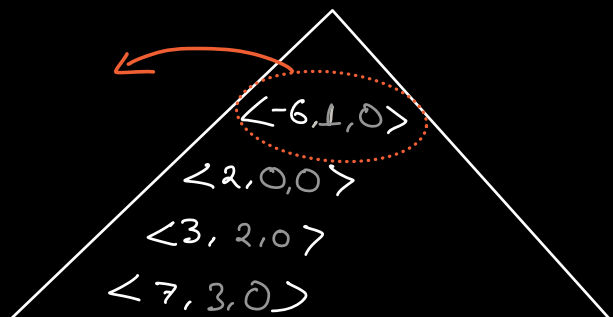
Approach 2

- Insert all elements in an array $O(NM)$ & convert it to a min heap.
- Pop out one-by-one. $O(NM \log NM)$

TC: $O(NM \log NM)$

	0	1	2	3	4
0	2	7	10	17	25
1	-6	0	1	8	11
2	3	4	6	14	21
3	7	10	14	19	23

$N \times M$



Class Data {

int val;

int r;

int c;

Data(val, r, c) {

this.val = val;

this.r = r;

this.c = c;

}

}

→ Remove min from heap

{V, r, c}

→ A[i] = V ; i++;

→ heap.add (new Data (M[r][c+1], r, c+1))

Max size of heap → N

TC : $O(NM \log N)$

SC : $O(N)$

Extra

Q
Amazon

Given 2 sorted arrays.

print k smallest sum pairs.

A : ⁰2 ¹5 ²8 ³11 ⁴13 → N

B : ⁰3 ¹7 ²9 ³12 ⁴15 ⁵16 ⁶20 → M K=5

A	B	
i	j	A[i] + B[j]
0, 0		5
1, 0		8
0, 1		9
0, 2		11
2, 0		11

Approach 1

→ Get sum of all possible pairs

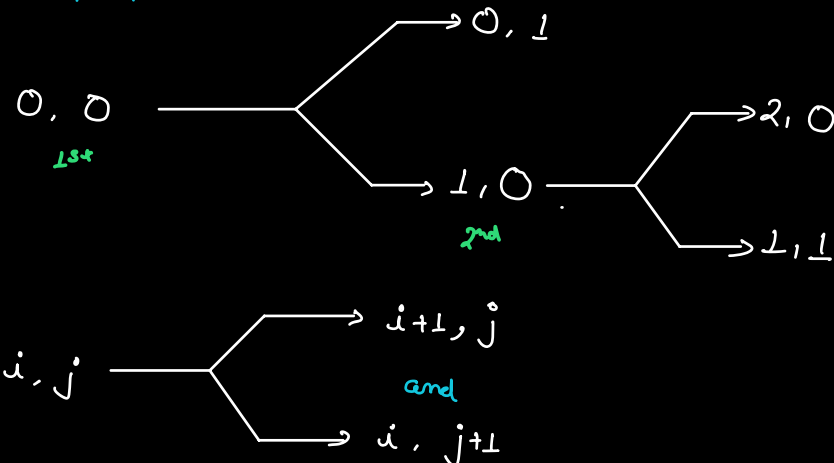
→ Sort & return first K

TC: $O(NM) + O(NM \log NM)$

$A : \overset{0}{2} \overset{1}{5} \overset{2}{8} \overset{3}{11} \overset{4}{13} \rightarrow N$
 $B : \overset{0}{3} \overset{1}{7} \overset{2}{9} \overset{3}{12} \overset{4}{15} \overset{5}{16} \overset{6}{20} \rightarrow M$

Smallest Sum

Pair



TC: $O(K \log K)$

$1 \xrightarrow{2-1} 2 \xrightarrow{2-1} 3 \xrightarrow{2-1} 4 \dots \rightarrow K$

PriorityQueue <Data> pq = new PriorityQueue <Data>();

Data Obj1 = new Data(0, 0, A(0)+B(0));

pq.add(obj1); ? $\longrightarrow$ Error!

```
Class Data implements Comparable<Data> {
```

```
    int val;
```

```
    int n;
```

```
    int c;
```

```
    Data(val, n, c) {
```

```
        this.val = val;
```

```
        this.n = n;
```

```
        this.c = c;
```

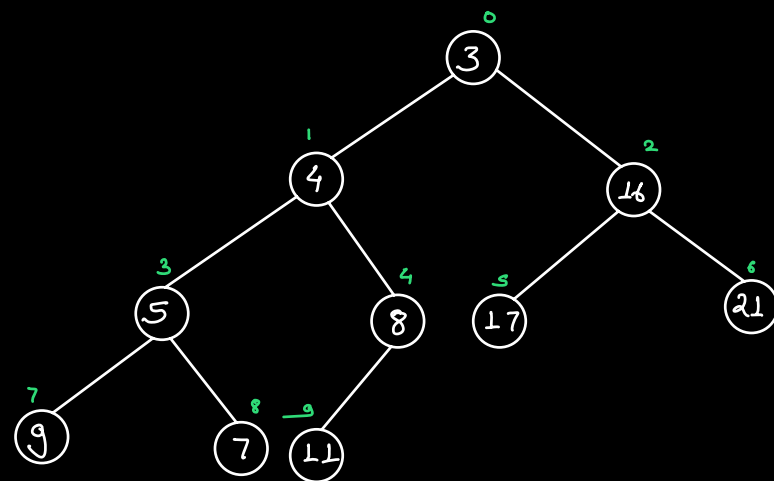
```
    }
```

```
    public int compareTo(Data o) {
```

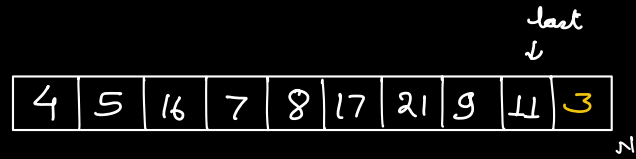
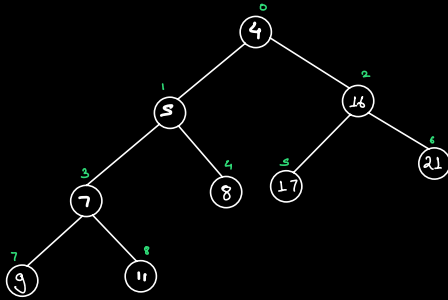
```
        return this.val - o.val;
```

```
    }
```

```
}
```

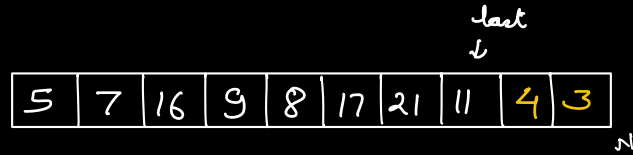
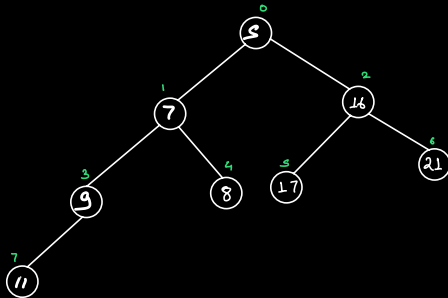


~

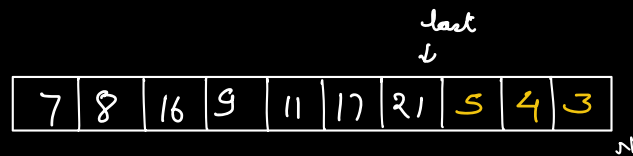
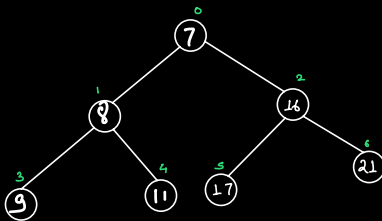


↓
delete Min()

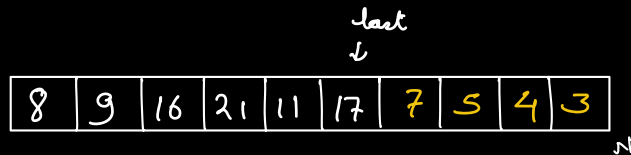
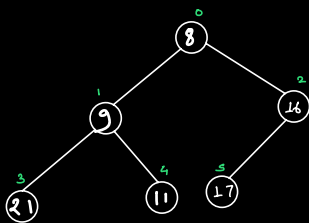
$O(\log N)$



↓
delete Min()



↓
delete Min()



↓
delete Min()

⋮

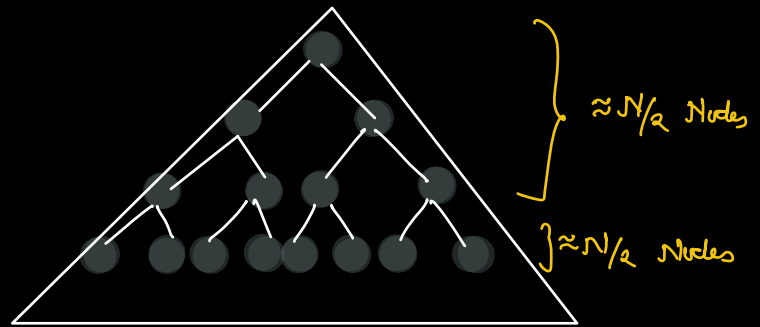
N Times



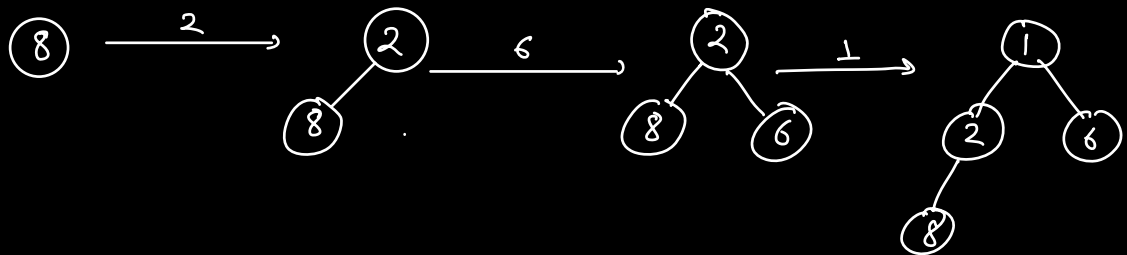
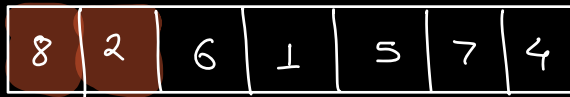
Heap Sort

TC : $O(N) + O(N \log N)$

SC : $O(1)$



last
↓



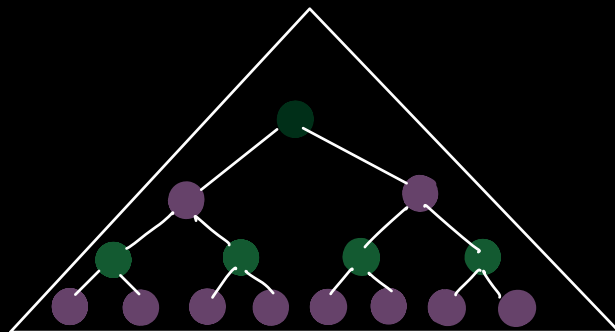
After inserting $N/2$ nodes

Size of heap $\longrightarrow N/2$
ht of heap $\longrightarrow \approx \log N$

For remaining $N/2$ nodes $\longrightarrow \log N$ iterations per insertion

TC: $O(N \log N)$

$N/2 \longrightarrow 0$ iterations
 $N/4 \longrightarrow 1$ iteration
 $N/8 \longrightarrow 2$ iterations
 $N/16 \longrightarrow 3$ iterations
 $\vdots$
 $1 \longrightarrow \log N$ iterations



$$\text{Total no. of iterations} = \frac{N}{2} \times 0 + \frac{N}{4} \times 1 + \frac{N}{8} \times 2 + \dots + 1 \times \log N$$

$$= \frac{N}{2^1} \times 0 + \frac{N}{2^2} \times 1 + \frac{N}{2^3} \times 2 + \frac{N}{2^4} \times 3 + \dots$$

$$= \sum_{d=0}^{\log N} \frac{N}{2^{d+1}} \times d$$

$$= \frac{N}{2} \left[\sum_{d=0}^{\log N} \frac{d}{2^d} \right] \quad \log N \rightarrow \infty$$

$$= \frac{N}{2} \times \left(\sum_{d=0}^{\infty} \frac{d}{2^d} \right) \rightarrow 2$$

$$\left[\frac{0}{2^0} + \frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \dots \right] \Rightarrow 2$$

$$= \frac{N}{2} \times 2$$

$$= N$$

TC: O(N)

